Practical 1 Ml Lab

Name : Ishan Manekar

Class : A_A3_45

In [1]:
```
import pandas as pd #data science
import numpy as np #calculations
#visualization
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:
```
# loading data set

df = pd.read_csv("titanic.csv")

df.head()
```

Out[2]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Emba
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	

In [3]:
```
# ps : you haev to apply ml to identify the rate of survival
```

In [4]:
```
df.tail()
```

Out[4]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75	NaN	Q

In [5]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object 
 11  Embarked     889 non-null    object 
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

In [6]:
```
df.describe()
```

Out[6]:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

In [7]:
```
df.isnull().sum()
```

Out[7]:
```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

In [8]:
```
 #handle missing values
df["Age"].fillna(df["Age"].median(), inplace = True)
```

```
C:\Users\HP\AppData\Local\Temp\ipykernel_15916\807133083.py:2: FutureWarning: A value is trying to be set on a co
of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on wh
we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'
df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df["Age"].fillna(df["Age"].median(), inplace = True)
```

In [9]:
```
df["Embarked"].fillna(df["Embarked"].mode()[0], inplace = True)
```

```
C:\Users\HP\AppData\Local\Temp\ipykernel_15916\1971420511.py:1: FutureWarning: A value is trying to be set on a c
of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on wh
we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'
df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df["Embarked"].fillna(df["Embarked"].mode()[0], inplace = True)
```

In [10]:
```
df.isnull().sum()
```

Out[10]:
```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age              0
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         0
dtype: int64
```

In [11]:
```
df.drop("Cabin" , axis=1, inplace =True) # droping the cabin feature because more than 60 percent of the data of
```

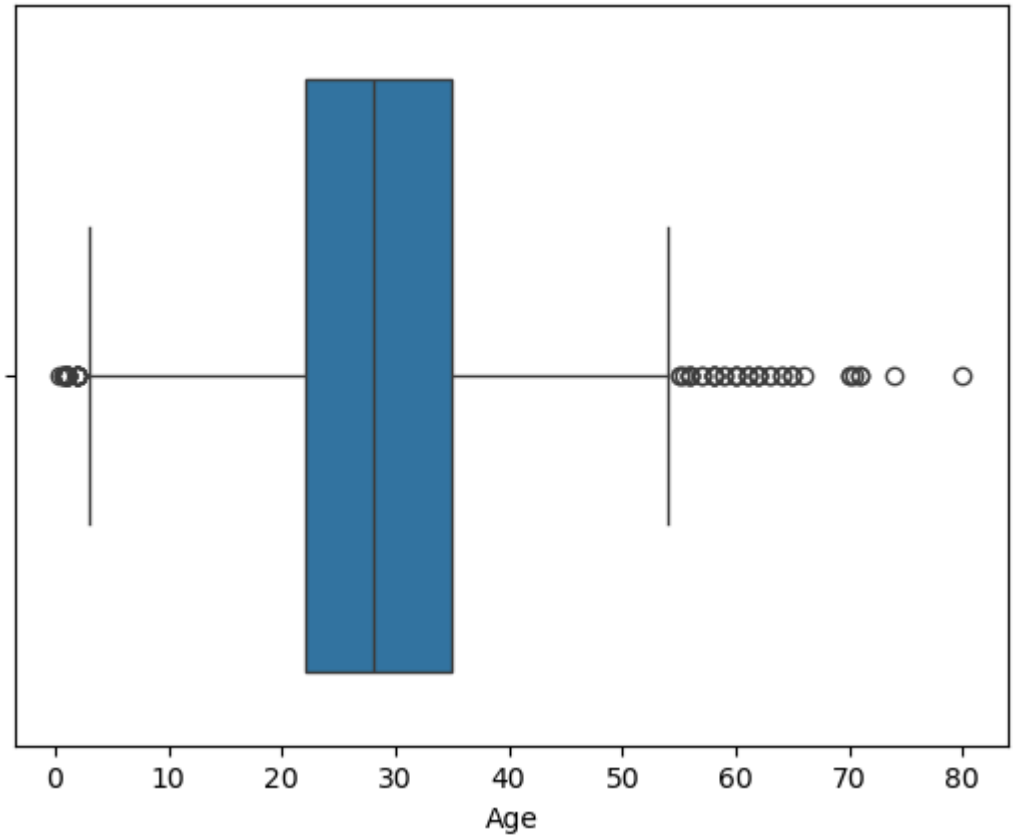
In [12]:
```
df.isnull().sum()
```

Out[12]:
```
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Embarked       0
dtype: int64
```

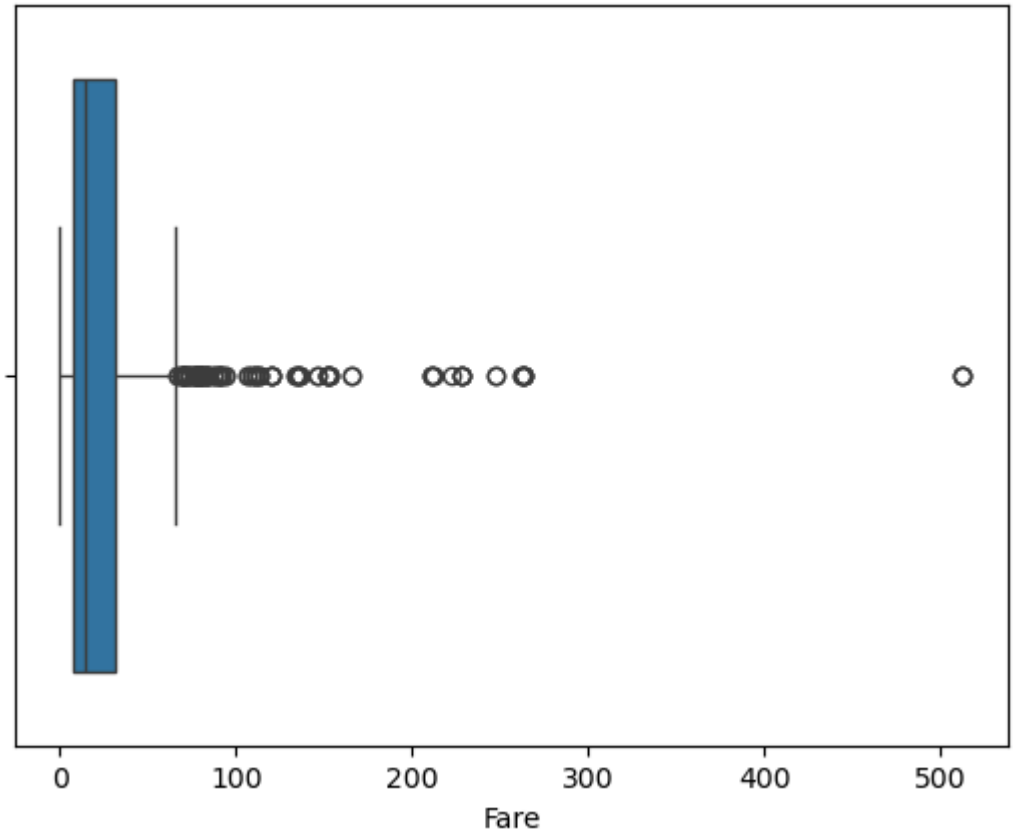
In [13]:
```
df.duplicated().sum()
```

Out[13]:
```
0
```

In [14]:
```
sns.boxplot(x=df["Age"])
plt.show()
```



In [15]:
```
sns.boxplot(x=df["Fare"])
plt.show()
```



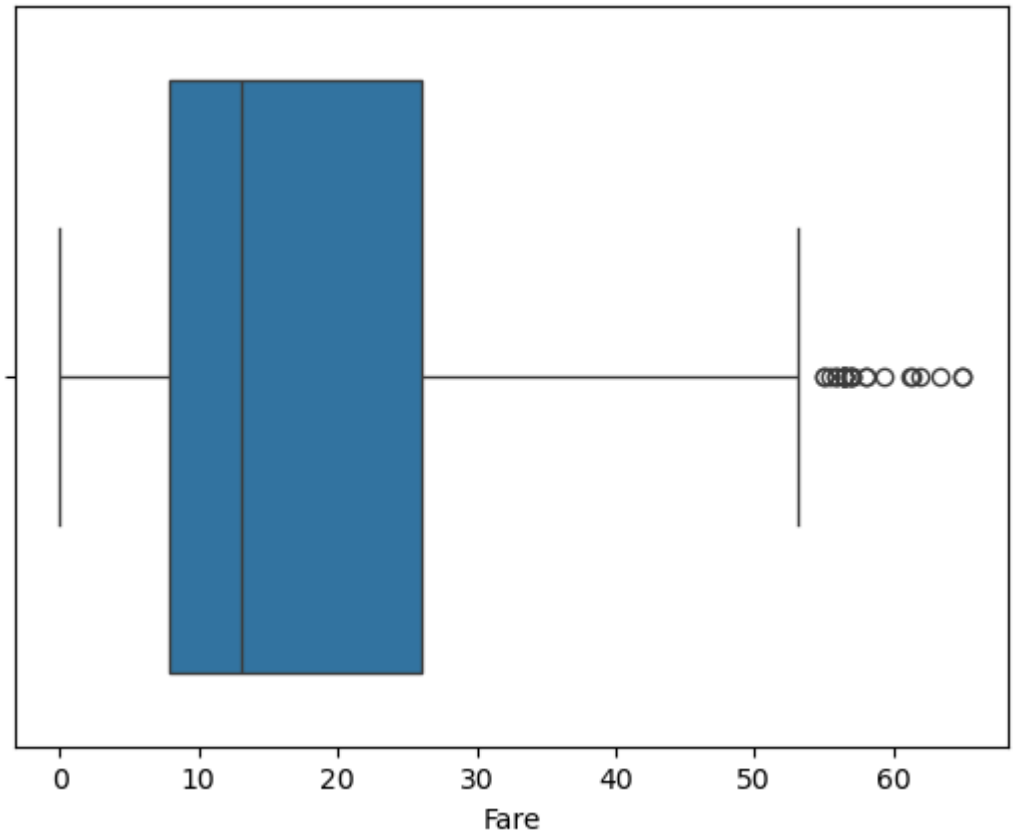
In [16]:
```
#outliers removing
Q1 = df["Fare"].quantile(0.25)
Q3 = df["Fare"].quantile(0.75)

IQR = Q3-Q1

lower = Q1 -1.5*IQR
higher =Q3+ 1.5*IQR

df= df[(df["Fare"] >= lower) & (df["Fare"] <= higher)]
```

In [17]:
```
sns.boxplot(x=df["Fare"])
plt.show()
```



In [18]:
```
#data encoding
df["Sex"] = df["Sex"].replace("male", 0)
df["Sex"] = df["Sex"].replace("female", 1)
```

```
C:\Users\HP\AppData\Local\Temp\ipykernel_15916\1490028317.py:3: FutureWarning: Downcasting behavior in `replace` 
deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer_obj
s(copy=False)`. To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
  df["Sex"] = df["Sex"].replace("female", 1)
```

In [19]:
```
df.head()
```

Out[19]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked
0	1	0	3	Braund, Mr. Owen Harris	0	22.0	1	0	A/5 21171	7.2500	S
2	3	1	3	Heikkinen, Miss. Laina	1	26.0	0	0	STON/O2. 3101282	7.9250	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	1	35.0	1	0	113803	53.1000	S
4	5	0	3	Allen, Mr. William Henry	0	35.0	0	0	373450	8.0500	S
5	6	0	3	Moran, Mr. James	0	28.0	0	0	330877	8.4583	Q

In [20]:
```
df["Embarked"].unique()
```

Out[20]:
```
array(['S', 'Q', 'C'], dtype=object)
```

In [21]:
```
df["Embarked"] = df["Embarked"].replace("S", 0)
df["Embarked"] = df["Embarked"].replace("C", 1)
df["Embarked"] = df["Embarked"].replace("Q", 2)
```

```
C:\Users\HP\AppData\Local\Temp\ipykernel_15916\3748634860.py:3: FutureWarning: Downcasting behavior in `replace` 
deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer_obj
s(copy=False)`. To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
  df["Embarked"] = df["Embarked"].replace("Q", 2)
```

In [22]:
```
df.head()
```

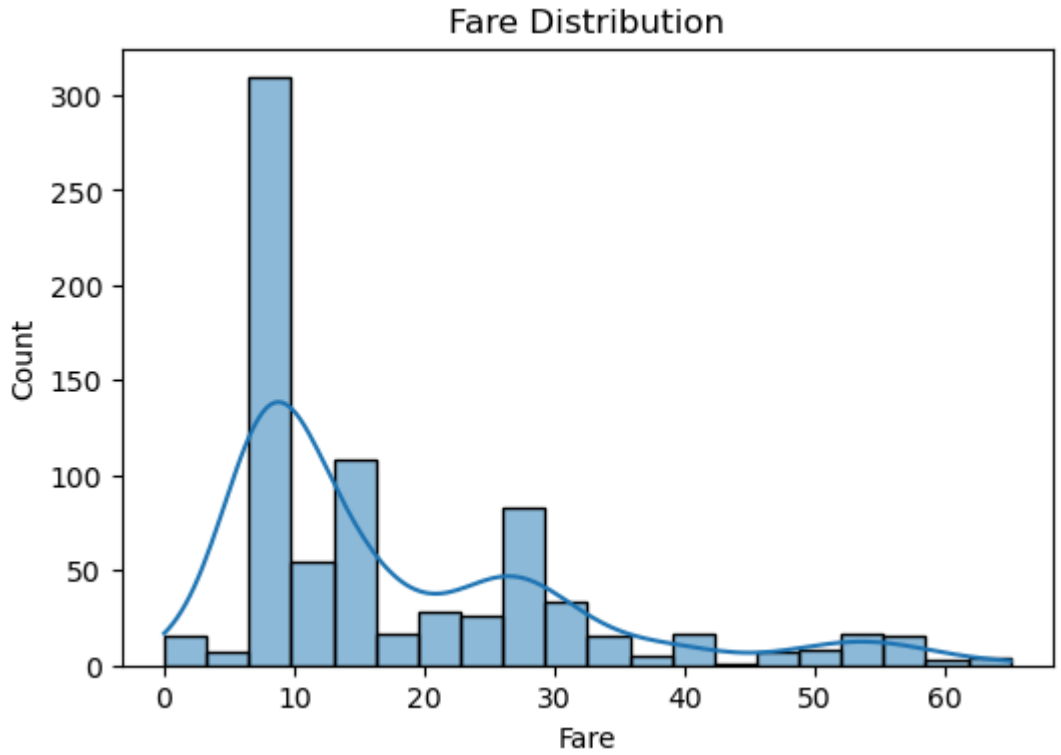
Out[22]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked
0	1	0	3	Braund, Mr. Owen Harris	0	22.0	1	0	A/5 21171	7.2500	0
2	3	1	3	Heikkinen, Miss. Laina	1	26.0	0	0	STON/O2. 3101282	7.9250	0
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	1	35.0	1	0	113803	53.1000	0
4	5	0	3	Allen, Mr. William Henry	0	35.0	0	0	373450	8.0500	0
5	6	0	3	Moran, Mr. James	0	28.0	0	0	330877	8.4583	2

In [23]:
```
#Univariate Analysis Fare

plt.figure(figsize=(6,4))
sns.histplot(df["Fare"], bins=20, kde=True)

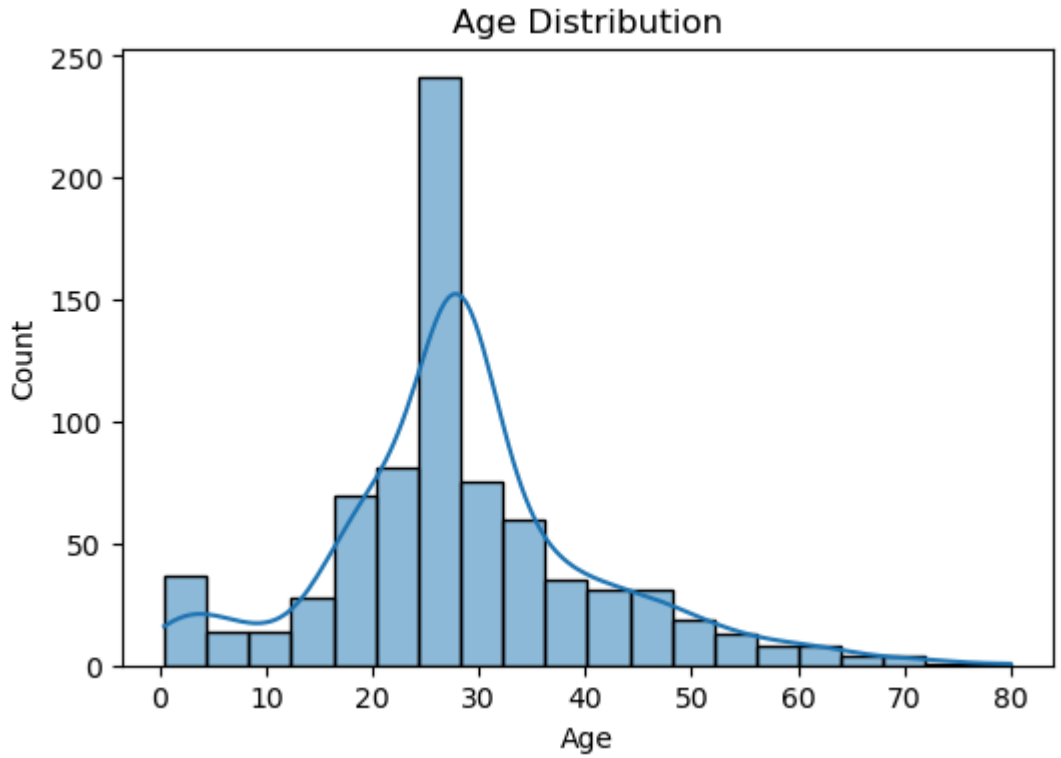
plt.title("Fare Distribution")
plt.show()
```



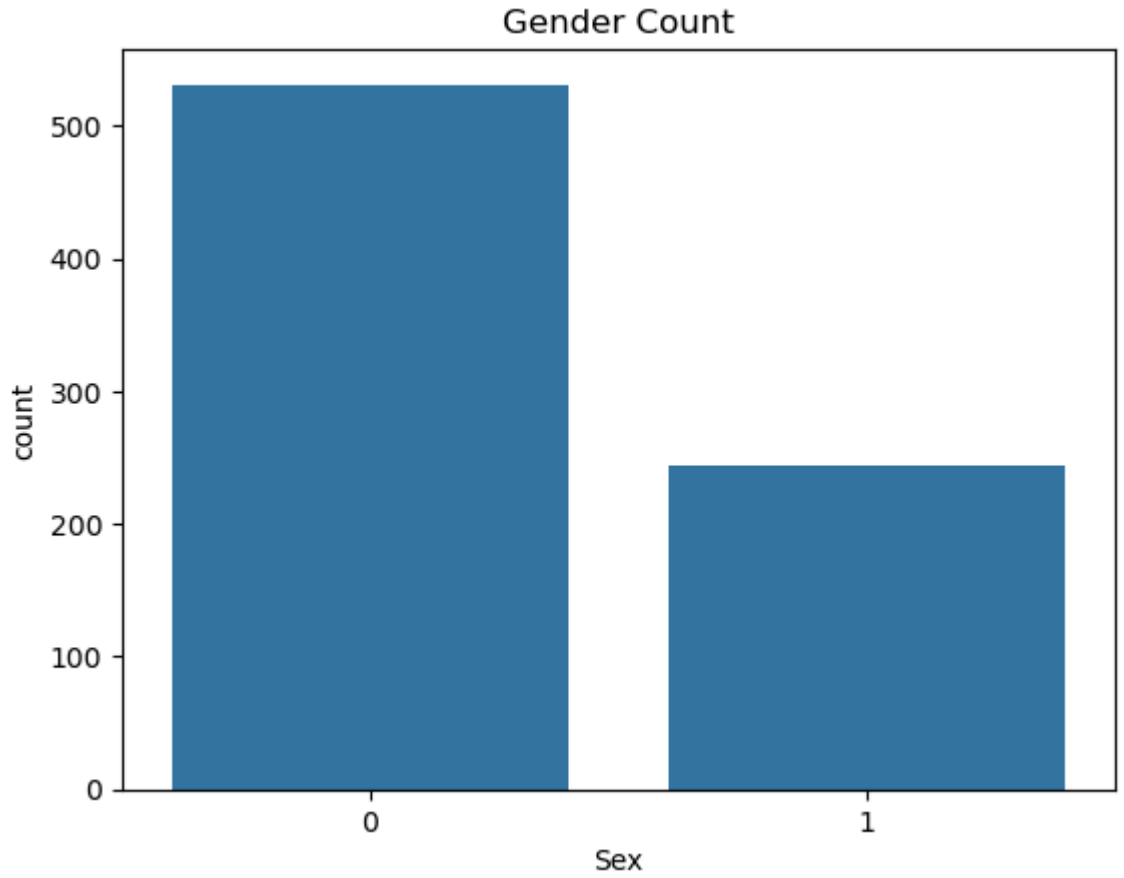
In [24]:
```
#Univariate Analysis

plt.figure(figsize=(6,4))
sns.histplot(df["Age"], bins=20, kde=True)

plt.title("Age Distribution")
plt.show()
```



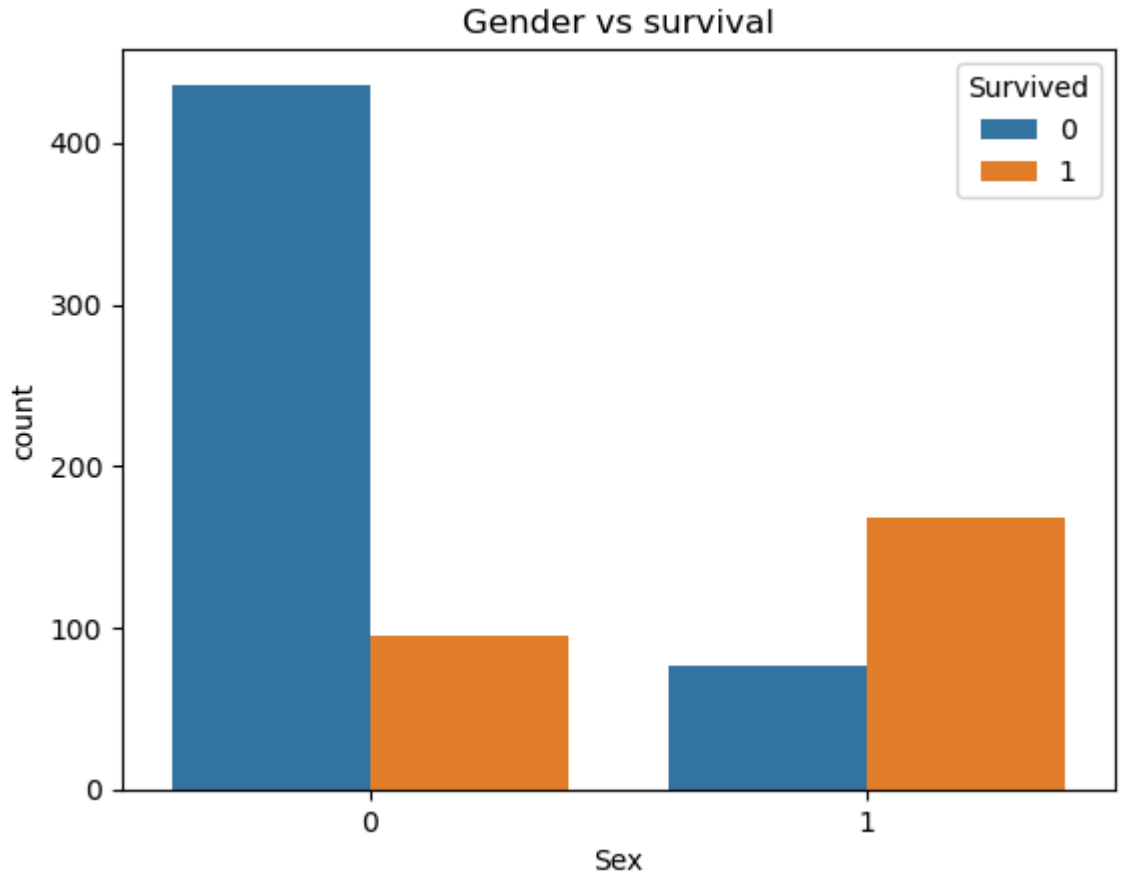
In [25]:
```
sns.countplot(x ="Sex", data=df)
plt.title("Gender Count")
plt.show()
```



In [26]:
```
#bivariate Analysis

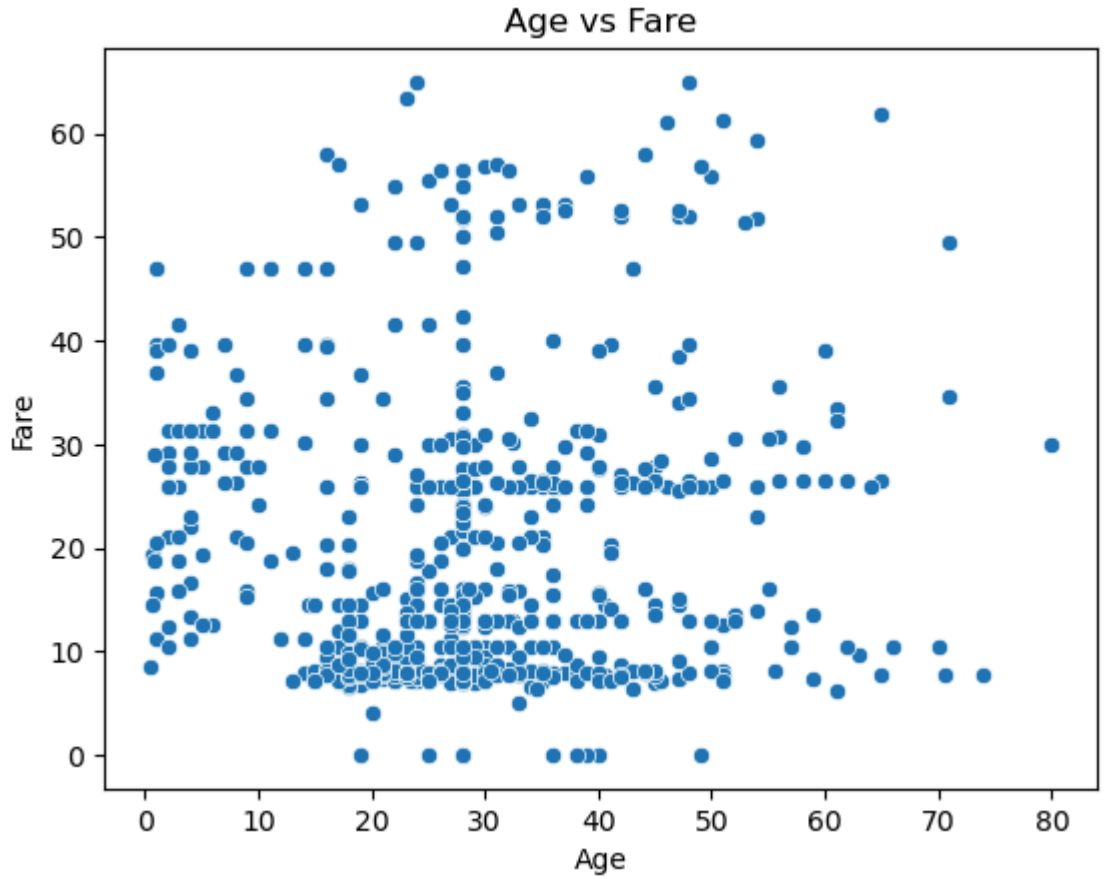
sns.countplot(x="Sex", hue="Survived", data =df)

plt.title("Gender vs survival")
plt.show()
```



In [27]:
```
sns.scatterplot(x="Age", y ="Fare", data =df)

plt.title("Age vs Fare")
plt.show()
```



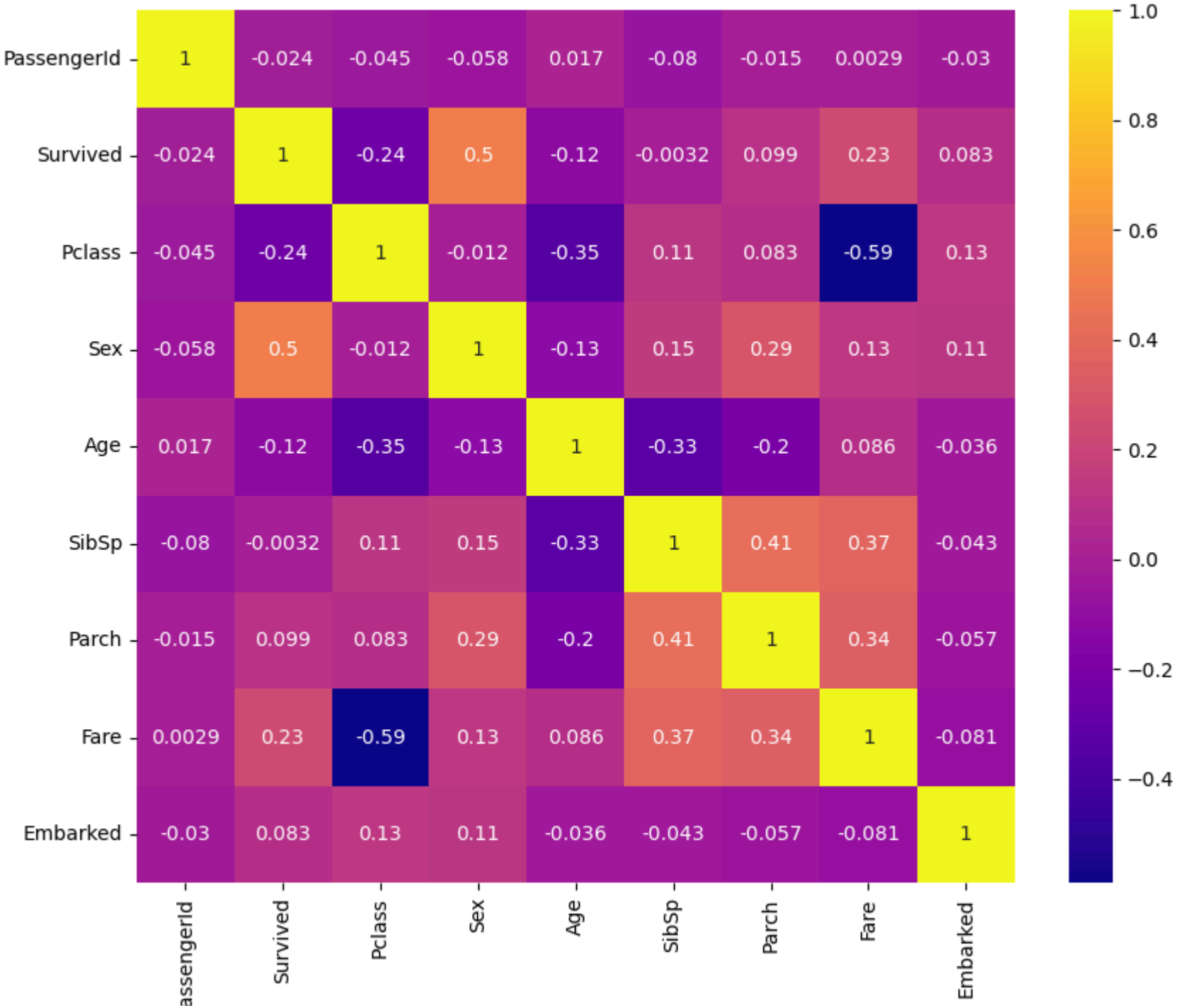
In [28]:
```
plt.figure(figsize=(10,8))

numeric_df = df.select_dtypes(include = ['number'])

sns.heatmap(numeric_df.corr(),
            annot =True,
            cmap='plasma')

plt.show()

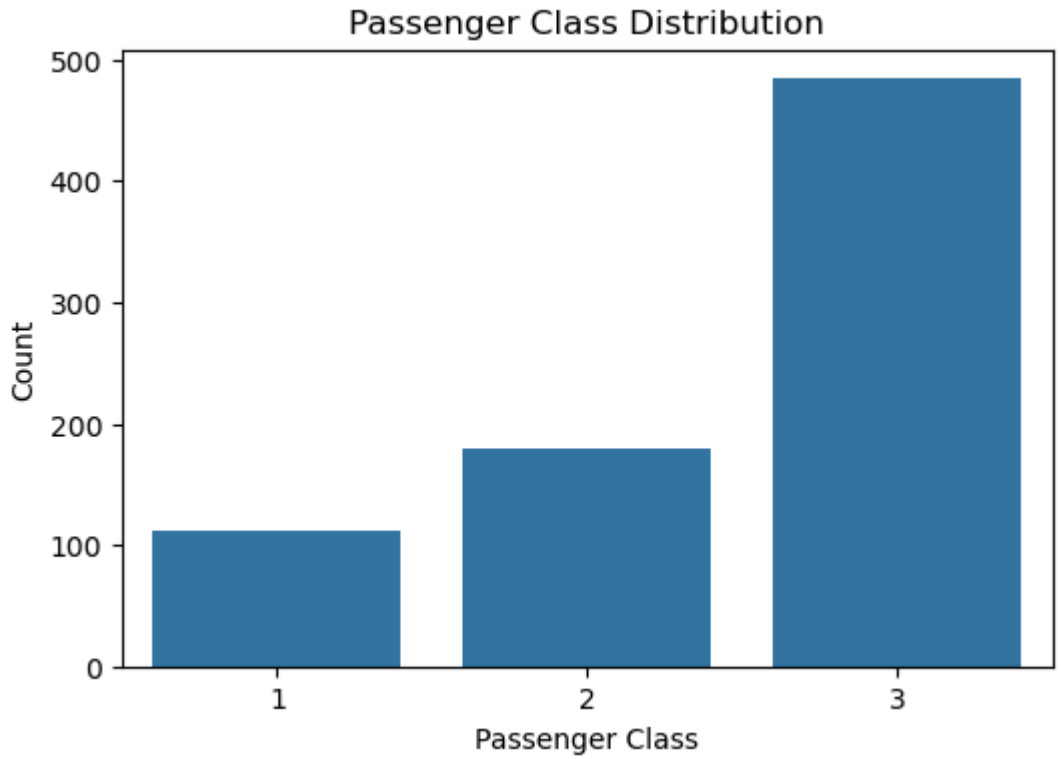
#virdis, plasma, inferno, magma, cividis ye sab colours hai seaborn ke
```


PassengerId 1 -0.024 -0.045 -0.058 0.017 -0.08 -0.015 0.0029 -0.03
Survived -0.024 1 -0.24 0.5 -0.12 -0.0032 0.099 0.23 0.083
Pclass -0.045 -0.24 1 -0.012 -0.35 0.11 0.083 -0.59 0.13
Sex -0.058 0.5 -0.012 1 -0.13 0.15 0.29 0.13 0.11
Age 0.017 -0.12 -0.35 -0.13 1 -0.33 -0.2 0.086 -0.036
SibSp -0.08 -0.0032 0.11 0.15 -0.33 1 0.41 0.37 -0.043
Parch -0.015 0.099 0.083 0.29 -0.2 0.41 1 0.34 -0.057
Fare 0.0029 0.23 -0.59 0.13 0.086 0.37 0.34 1 -0.081
Embarked -0.03 0.083 0.13 0.11 -0.036 -0.043 -0.057 -0.081 1
assengerId
Survived
Pclass
Sex
Age
SibSp
Parch
Fare
Embarked
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4

In [29]:
```
#count Plot (categorical data)

plt.figure(figsize=(6,4))
sns.countplot(x="Pclass", data =df)

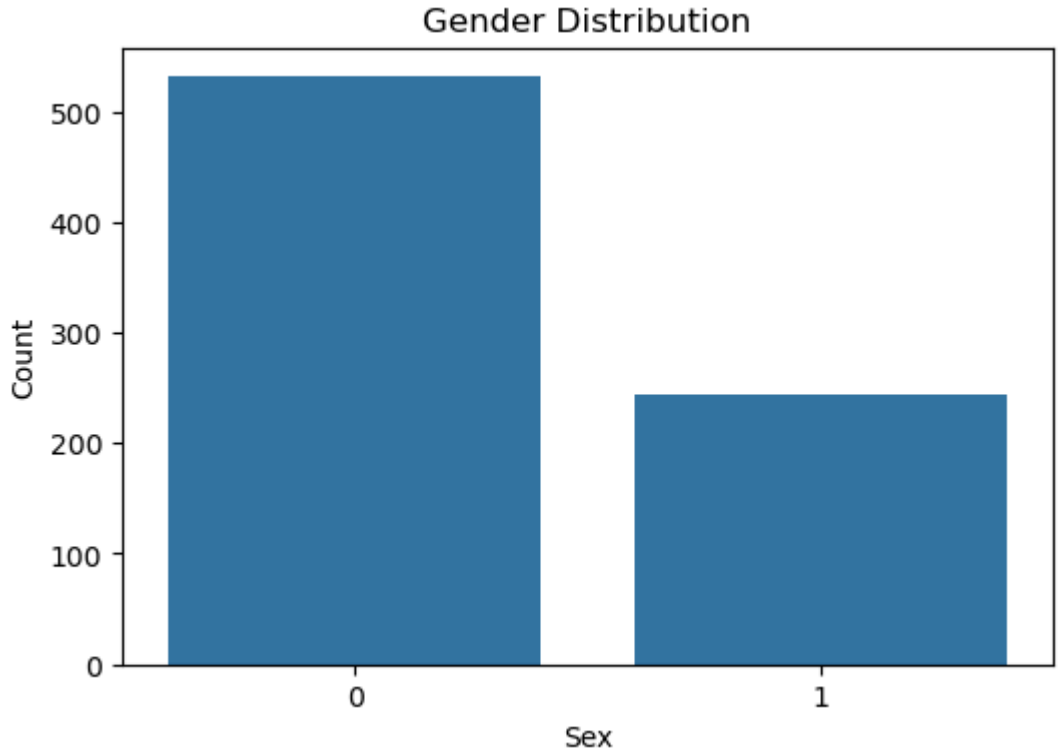
plt.title("Passenger Class Distribution")
plt.xlabel("Passenger Class")
plt.ylabel("Count")
plt.show()
```



In [30]:
```
#count Plot (categorical data)

plt.figure(figsize=(6,4))
sns.countplot(x="Sex", data =df)

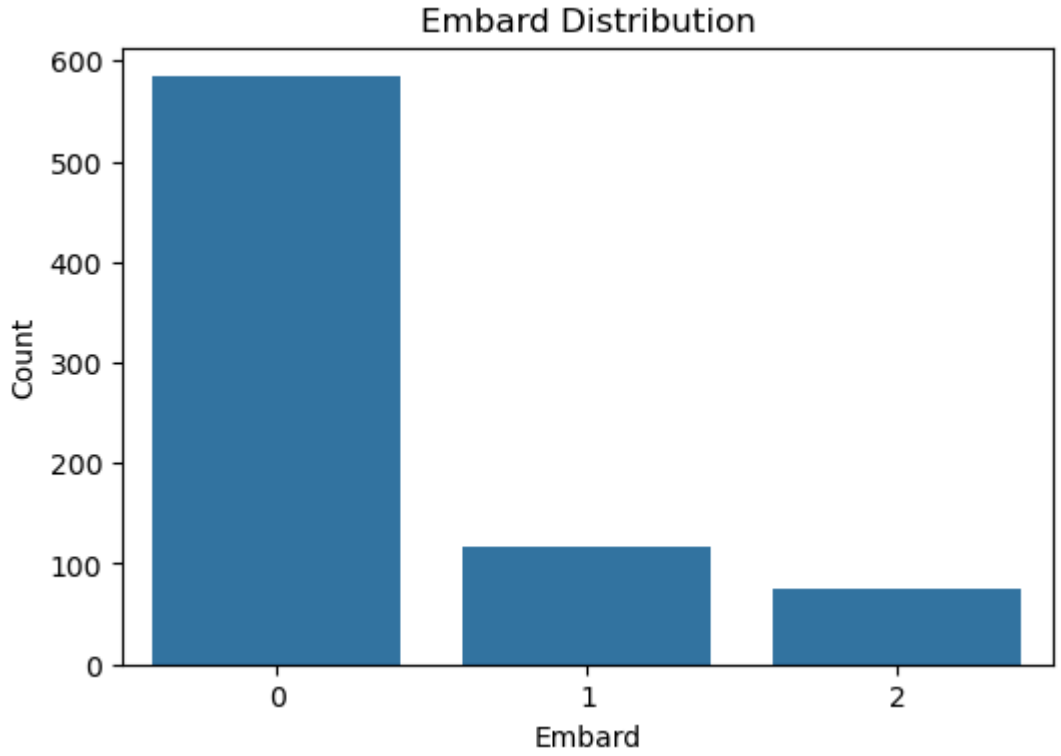
plt.title("Gender Distribution")
plt.xlabel("Sex")
plt.ylabel("Count")
plt.show()
```



In [31]:
```
#count Plot (categorical data)

plt.figure(figsize=(6,4))
sns.countplot(x="Embarked", data =df)

plt.title("Embard Distribution")
plt.xlabel("Embard")
plt.ylabel("Count")
plt.show()
```



Scaling

In [32]:
```
#feature scaling
X = df.drop("Survived", axis =1)
y = df["Survived"]
```

In [35]:
```
X = X.drop(["PassengerId", "Name", "Ticket"], axis =1)
```

In [36]:
```
#standard scaling

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X[["Age", "Fare"]] = scaler.fit_transform(X[["Age", "Fare"]])
```

In [37]:
```
X.head(15)
```

Out[37]:

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	3	0	-0.528321	1	0	-0.779117	0
2	3	1	-0.215182	0	0	-0.729373	0
3	1	1	0.489381	1	0	2.599828	0
4	3	0	0.489381	0	0	-0.720161	0
5	3	0	-0.058613	0	0	-0.690071	2
6	1	0	1.976792	0	0	2.508630	0
7	3	0	-2.094017	3	1	0.239725	0
8	3	1	-0.136897	0	2	-0.492935	0
9	2	1	-1.154600	1	0	0.902677	1
10	3	1	-1.937447	1	1	-0.082693	0
11	1	1	2.289931	0	0	0.643209	0
12	3	0	-0.684891	0	0	-0.720161	0
13	3	0	0.802520	1	5	0.991421	0
14	3	1	-1.154600	0	0	-0.734590	0
15	2	1	2.055077	0	0	-0.134280	0

In [38]:
```
#train test split

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X,y, test_size =0.20, random_state = 42)
```

In [39]:
```
print("Training Data : ", X_train.shape)

print("Testing Data : ", X_test.shape)
```

```
Training Data :  (620, 7)
Testing Data :  (155, 7)
```

In [40]:
```
print(X_train.head())
```

```
     Pclass  Sex       Age  SibSp  Parch      Fare  Embarked
147       3    1 -1.546024      2      2  1.219877         0
868       3    0 -0.058613      0      0 -0.613302         0
60        3    0 -0.528321      0      0 -0.780650         1
468       3    0 -0.058613      0      0 -0.744112         2
777       3    1 -1.859163      0      0 -0.394057         0
```

In [ ]:
```

```